

tell me step by step how to create cluster of mongodb on 3 pcs

Setting up a 3-node MongoDB cluster usually means creating a **replica set** (for redundancy and basic high availability). I'll walk you through it clearly.

What you're building

A MongoDB replica set = 3 machines:

- 1 Primary (handles writes)
- 2 Secondary (replicate data)

Prerequisites

On all 3 PCs:

- Install MongoDB (same version on all)
- Ensure:
 - All machines can reach each other (ping / same network)
 - Static IPs or stable hostnames
 - Ports open (default 27017)

Example:

PC1 → 192.168.1.10

PC2 → 192.168.1.11

PC3 → 192.168.1.12

Step 1: Configure MongoDB on each PC

Edit the MongoDB config file (`mongod.conf`)

Add / modify:

```
replication:
  replSetName: "rs0"

net:
  bindIp: 0.0.0.0
  port: 27017
```

👉 Important:

- `replSetName` must be identical on all 3 machines

▶ Step 2: Start MongoDB on all machines

Run on each PC:

Bash

🔗 Step 3: Initialize the replica set

Connect to one machine (PC1):

Bash

Then run:

```
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "192.168.1.10:27017" },
    { _id: 1, host: "192.168.1.11:27017" },
    { _id: 2, host: "192.168.1.12:27017" }
  ]
})
```

Step 4: Check status

JavaScript

You should see:

- One node → PRIMARY
- Two nodes → SECONDARY

Step 5: Test replication

On PRIMARY:

```
use testdb
```

JavaScript

On SECONDARY:

```
rs.secondaryOk()
```

JavaScript

👉 You should see the same data.

(Optional but Recommended) Step 6: Enable authentication

Create admin user:

```
use admin
db.createUser({
  user: "admin",
```

```
pwd: "strongpassword",
roles: [ { role: "root", db: "admin" } ]
```

```
JavaScript
```

Then enable security in config:

```
security:
```

```
YAML
```

Restart all nodes.

Step 7: (Optional) Set priorities

Control which node becomes primary:

```
rs.reconfig({
  _id: "rs0",
  members: [
    { _id: 0, host: "192.168.1.10:27017", priority: 2 },
    { _id: 1, host: "192.168.1.11:27017", priority: 1 },
    { _id: 2, host: "192.168.1.12:27017", priority: 1 }
  ]
})
```

Key Tips

- Always use odd number of nodes (you did 👍)
- Minimum 3 nodes = quorum for elections
- Use SSD + good network for better performance
- Time sync (NTP) is important

Bonus: If you meant *sharding cluster*

Replica set ≠ full cluster scaling. If you want:

- Horizontal scaling
- Massive datasets

Then you need:

- Config servers
- Shards (each is a replica set)
- Mongos router

👉 Tell me if you want that — it's a bigger setup.

If you share your OS (Windows/Linux) or whether you're using Docker, I can tailor this into an exact copy-paste setup.



which cluster is easy mongodb cluster or postgres cluster

Short answer: **MongoDB clustering is generally easier to set up than PostgreSQL clustering**—especially for a basic 3-node setup.

But the “why” matters 👉

🟢 MongoDB (Replica Set) — Easier 👍

With MongoDB:

- Built-in replication (no extra tools)
- Automatic failover out of the box
- Simple configuration (`rs.initiate()` and done)
- No need for external orchestration

What you do:

- Install MongoDB on 3 machines
- Add `repSetName`
- Run one command → cluster is live

👉 That's it for a working HA setup.

🟦 PostgreSQL Cluster — More Complex ⚠️

With PostgreSQL:

- Replication exists, BUT:
 - No built-in automatic failover (by default)
- You need extra tools like:
 - Patroni
 - repmgr
 - Pgpool-II
- Requires:
 - WAL configuration
 - Base backups
 - Replication slots
 - External coordination (like etcd)

What you end up managing:

- Leader election (not automatic by default)
- Failover logic
- More config files + moving parts

⚖️ Direct Comparison

Feature	MongoDB	PostgreSQL
Setup difficulty	✅ Easy	❌ Medium–Hard
Built-in replication	✅ Yes	✅ Yes
Auto failover	✅ Yes	❌ Needs tools
Commands needed	Few	Many
External tools required	❌ No	✅ Yes



Reality Check

- MongoDB = easier to get running
- PostgreSQL = more control, but more engineering effort



Verdict



If your goal is:

- Learning clustering quickly → **MongoDB wins**
- Production-grade relational system → PostgreSQL (but expect complexity)

If you want, I can:

- [Show a PostgreSQL 3-node cluster step-by-step](#)
- Or [compare performance + use cases \(when to choose which\)](#)



now u tell me mongodb clustering on single pc of 3 nodes
without docker

Yes—you can absolutely simulate a 3-node MongoDB cluster on a **single PC without Docker**. This is commonly done for learning using multiple ports and data directories.

Here's a clean, step-by-step way to do it with [MongoDB](#).



What you'll create

On one machine:

- 3 MongoDB instances (each acts like a separate node)
- Each runs on a different port
- Combined into a **replica set**

Step 1: Create directories

Create separate folders for each node:

```
mkdir -p ~/mongo-cluster/node1
mkdir -p ~/mongo-cluster/node2
```

Bash

Step 2: Create config files

node1.conf

```
storage:
  dbPath: /home/youruser/mongo-cluster/node1

net:
  bindIp: 127.0.0.1
  port: 27017

replication:
  replSetName: rs0
```

node2.conf

```
storage:
  dbPath: /home/youruser/mongo-cluster/node2

net:
  bindIp: 127.0.0.1
  port: 27018

replication:
  replSetName: rs0
```

node3.conf


```
storage:  
  dbPath: /home/youruser/mongo-cluster/node3
```

```
net:  
  bindIp: 127.0.0.1  
  port: 27019
```

```
replication:  
  replSetName: rs0
```

👉 Key idea:

- Same `replSetName`
- Different **ports** + **dbPath**

▶ Step 3: Start all 3 MongoDB instances

Open 3 terminals and run:

Bash

Bash

Bash

🔗 Step 4: Initialize replica set

Connect to node1:

Bash

Run:

```
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "127.0.0.1:27017" },
    { _id: 1, host: "127.0.0.1:27018" },
    { _id: 2, host: "127.0.0.1:27019" }
  ]
})
```

Step 5: Verify

JavaScript

You should see:

- 1 PRIMARY
- 2 SECONDARY

Step 6: Test replication

On primary:

```
use testdb
```

JavaScript

Connect to another node:

Bash

Then:

```
rs.secondaryOk()
```

JavaScript

Important Notes

This is for learning only

- All nodes are on same machine → no real fault tolerance
- If PC crashes → everything goes down

Good for:

- Practicing replica sets
- Testing failover behavior
- Development

Bonus: Simulate failure

Kill primary process:

Bash

Watch:

- One secondary becomes new PRIMARY automatically

Cleanup tip

To restart fresh:

Bash

If you want next level:

- Add **authentication + keyFile**
- Convert this into a **mini sharded cluster**
- Or run it on Windows (slightly different paths)

Just tell me 👍



i dont have anything tell me step by step how to create cluster

Got it—you're starting from zero. I'll walk you through **everything from installation to a working 3-node MongoDB cluster on ONE PC** (no Docker, beginner-friendly).

We'll use MongoDB and build a replica set.



STEP 0: What you need

- A single PC (Linux / WSL recommended)
- Internet connection
- Terminal access



I'll assume **Ubuntu/Linux**. If you're on Windows, tell me and I'll adjust.



STEP 1: Install MongoDB

Run:

```
sudo apt update
```

```
Bash
```

Check install:

Bash

STEP 2: Create folders for 3 nodes

```
mkdir -p ~/mongo-cluster/node1  
mkdir -p ~/mongo-cluster/node2
```

Bash

STEP 3: Create config files

Create 3 config files:

node1.conf

Bash

Paste:

```
storage:  
  dbPath: /home/$USER/mongo-cluster/node1  
  
net:  
  bindIp: 127.0.0.1  
  port: 27017  
  
replication:  
  replSetName: rs0
```

node2.conf

Bash

```
storage:
  dbPath: /home/$USER/mongo-cluster/node2

net:
  bindIp: 127.0.0.1
  port: 27018

replication:
  replSetName: rs0
```

node3.conf

Bash

```
storage:
  dbPath: /home/$USER/mongo-cluster/node3

net:
  bindIp: 127.0.0.1
  port: 27019

replication:
  replSetName: rs0
```



STEP 4: Start 3 MongoDB servers

Open 3 separate terminals.

Terminal 1:

Bash

Terminal 2:

Bash

Terminal 3:

Bash

👉 Keep all 3 running.

STEP 5: Connect and create cluster

Open a NEW terminal:

Bash

Run this:

```
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "127.0.0.1:27017" },
    { _id: 1, host: "127.0.0.1:27018" },
    { _id: 2, host: "127.0.0.1:27019" }
  ]
})
```



STEP 6: Check cluster

JavaScript



You should see:

- 1 PRIMARY
- 2 SECONDARY



STEP 7: Test it

Insert data:

```
use testdb
```

JavaScript

Now connect to another node:

Bash

Then:

```
rs.secondaryOk()
```

JavaScript



If you see the data → cluster works 🎉



STEP 8: Test failover (important)

Kill the primary terminal (Ctrl+C)

Then run:

JavaScript

👉 One secondary becomes PRIMARY automatically.

💡 Common mistakes

- ❌ Ports same → won't work
- ❌ replSetName different → won't join
- ❌ not running all 3 nodes → fails

🏁 You now have:

- A working 3-node MongoDB cluster
- Automatic failover
- Replication

If you want next steps, I can show you:

- 🔒 Add username/password (real production setup)
- ⚡ Run all nodes in background (systemd)
- 🌐 Convert this into multi-PC cluster

Just tell me 👍



windows

Alright—here's a **clean, from-scratch guide** to run a 3-node MongoDB cluster on a single **Windows PC** (no Docker, no prior setup).

We'll use MongoDB and create a **replica set**.



STEP 1: Download & Install MongoDB

1. Go to:



<https://www.mongodb.com/try/download/community>

2. Choose:

- Version: Latest
- Platform: Windows
- Package: MSI

3. Install:

- Click **Next** → **Complete**
- ☒ Install MongoDB as a Service (optional but fine)
- ☒ Install MongoDB Compass (optional)



STEP 2: Stop default MongoDB service

If installed as a service, stop it (important to avoid port conflict):

Open **Command Prompt as Administrator**:

```
bat
```



STEP 3: Create folders for 3 nodes

Open **Command Prompt**:

```
mkdir C:\mongo-cluster\node1
mkdir C:\mongo-cluster\node2
mkdir C:\mongo-cluster\node3

mkdir C:\mongo-cluster\config
.
```



STEP 4: Create config files

Open Notepad and create 3 files:

node1.cfg

Save at: C:\mongo-cluster\config\node1.cfg

```
storage:
  dbPath: C:\mongo-cluster\node1
```

```
net:
  bindIp: 127.0.0.1
  port: 27017
```

```
replication:
  replSetName: rs0
```

node2.cfg

```
storage:
  dbPath: C:\mongo-cluster\node2
```

```
net:
  bindIp: 127.0.0.1
  port: 27018
```

```
replication:
  replSetName: rs0
```

node3.cfg

```
storage:
  dbPath: C:\mongo-cluster\node3
```

```
net:
```

```
bindIp: 127.0.0.1  
port: 27019
```

```
replication:  
  replSetName: rs0
```

STEP 5: Start 3 MongoDB nodes

Open 3 separate Command Prompt windows

Window 1:



bat

Window 2:



bat

Window 3:



bat

 Keep all 3 windows running.

STEP 6: Initialize the cluster

Open a new Command Prompt:

bat

Then run:

```
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "127.0.0.1:27017" },
    { _id: 1, host: "127.0.0.1:27018" },
    { _id: 2, host: "127.0.0.1:27019" }
  ]
})
```

STEP 7: Check status

JavaScript

✓ You should see:

- 1 PRIMARY
- 2 SECONDARY

STEP 8: Test replication

Insert data:

```
use testdb
```

JavaScript

Now open another shell:

bat

Run:

```
rs.secondaryOk()
```

JavaScript

👉 Data should appear ✔

🌟 STEP 9: Test failover

Close the PRIMARY window (or Ctrl+C)

Then:

JavaScript

👉 One secondary becomes PRIMARY automatically

🧠 Important Notes

- This is **learning setup only**
- All nodes on same PC = no real fault tolerance
- Ports must be different (27017, 27018, 27019)

🏁 Done 🎉

You now have:

- A working MongoDB 3-node cluster
- Replication + failover

If you want next:

-  Add username/password (production-style)
-  Run nodes as Windows services (background)
-  Move this to 3 real machines

Just tell me 



```
C:\Program Files\MongoDB\Server\8.3\bin>cd
mongod.exe
The directory name is invalid.
```

You're getting that error because this command is wrong:

```
bat
```

 `cd` = **change directory**, but `mongod.exe` is a **file**, not a folder.

Correct way to run MongoDB

You don't "cd into" `mongod.exe`. You either:

Option 1: Run it directly (best)

From inside:

```
bat
```

Just run:

```
bat
```

✓ Option 2: Use full path (works from anywhere)

```
bat
```

✓ Option 3: First go to folder, then run

```
cd "C:\Program Files\MongoDB\Server\8.3\bin"
```

```
bat
```

💡 Why your error happened

You did:

```
bat
```

Windows tried to:

👉 Find a folder named `mongod.exe`

👉 It doesn't exist → ❌ error

🚀 What to do next

Open 3 command prompts and run:

```
bat
```


bat

bat

If you hit another error, paste it here—I'll fix it step-by-step with you.



```
:
{"code":26,"codeName":"NamespaceNotFound","errmsg":"
Unable to retrieve storageStats in $collStats stage :: caused
by :: Collection [local.oplog.rs] not found."},"stats":{},"cmd":
{"aggregate":"oplog.rs","cursor":{},"pipeline":[{"$collStats":
{"storageStats":
{"waitForLock":false,"numericOnly":true}}]},"$db":"local"}}
{"t":{"$date":"2026-05-06T22:05:17.905+05:30"},"s":"I",
"c":"-", "id":4939300, "ctx":"monitoring-keys-for-
HMAC","msg":"Failed to refresh key cache","attr":
{"error":"ReadConcernMajorityNotAvailableYet: Read
concern majority reads are currently not
possible."},"nextRefreshIntervalMillis":5200}}
{"t":{"$date":"2026-05-06T22:05:18.003+05:30"},"s":"W",
"c":"QUERY", "id":23799, "ctx":"ftdc","msg":"Aggregate
command executor error","attr":{"error":
{"code":26,"codeName":"NamespaceNotFound","errmsg":"
Unable to retrieve storageStats in $collStats stage :: caused
by :: Collection [local.oplog.rs] not found."},"stats":{},"cmd":
{"aggregate":"oplog.rs","cursor":{},"pipeline":[{"$collStats":
{"storageStats":
{"waitForLock":false,"numericOnly":true}}]},"$db":"local"}}
{"t":{"$date":"2026-05-06T22:05:19.006+05:30"},"s":"W",
```

```
"c":"QUERY", "id":23799, "ctx":"ftdc","msg":"Aggregate
command executor error","attr":{"error":
{"code":26,"codeName":"NamespaceNotFound","errmsg":
"Unable to retrieve storageStats in $collStats stage :: caused
by :: Collection [local.oplog.rs] not found."},"stats":{},"cmd":
{"aggregate":"oplog.rs","cursor":{},"pipeline":[{"$collStats":
{"storageStats":
{"waitForLock":false,"numericOnly":true}}]},"$db":"local"}}
{"t":{"$date":"2026-05-06T22:05:20.002+05:30"},"s":"W",
"c":"QUERY", "id":23799, "ctx":"ftdc","msg":"Aggregate
command executor error","attr":{"error":
{"code":26,"codeName":"NamespaceNotFound","errmsg":
"Unable to retrieve storageStats in $collStats stage :: caused
by :: Collection [local.oplog.rs] not found."},"stats":{},"cmd":
{"aggregate":"oplog.rs","cursor":{},"pipeline":[{"$collStats":
{"storageStats":
{"waitForLock":false,"numericOnly":true}}]},"$db":"local"}}
```

This error looks scary, but it actually tells you something very specific—and fixable.

! What the error means

Collection [local.oplog.rs] not found

👉 This means:

- Your MongoDB instance has **NOT** been initialized as a replica set yet

The `oplog.rs` collection is **created automatically only after** `rs.initiate()` runs successfully.

💡 Why you're seeing this

Right now:

- You started `mongod` with `replSetName`
- BUT you did **NOT** initialize the replica set yet

So MongoDB keeps trying to access replication data → fails → logs this warning

👉 This is normal at this stage.

✅ FIX (Do this now)

Step 1: Open a new terminal

Run:

```
bat
```

Step 2: Initialize replica set

Paste this:

```
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "127.0.0.1:27017" },
    { _id: 1, host: "127.0.0.1:27018" },
    { _id: 2, host: "127.0.0.1:27019" }
  ]
})
```

Step 3: Wait 5–10 seconds

Then run:

```
JavaScript
```

🎯 What should happen

- That error will **disappear**
- MongoDB will create:
 - `local.oplog.rs` 
- One node becomes:
 - PRIMARY
- Others:
 - SECONDARY

If you get error in `rs.initiate()`

Tell me exactly what it says—but common issues:

Node not running

→ Make sure all 3 `mongod` windows are open

Port mismatch

→ Ensure:

- 27017
- 27018
- 27019

Simple explanation

Before `rs.initiate()`

No oplog

No replication

Errors appear

After `rs.initiate()`

oplog created

replication active

errors gone

You're very close

That error actually means:

 Your setup is working, just missing one final step

If you want, next I can help you:

- Verify cluster visually
- Add authentication
- Or debug if `rs.initiate()` fails

Just paste the next output 👍



```
C:\Program Files\MongoDB\Server\8.3\bin>mongosh.exe -  
-port 27017  
'mongosh.exe' is not recognized as an internal or external  
command,  
operable program or batch file.
```

```
C:\Program Files\MongoDB\Server\8.3\bin>
```

Good—you're very close. This error just means **Windows can't find** `mongosh.exe` .

! Why this happens

bat

👉 Means:

- Either:
 - `mongosh` is **not installed**, OR
 - You're **not in the correct folder**, OR
 - It's not in PATH

✅ Step-by-step fix

Step 1: Check if `mongosh.exe` exists

Go to this folder manually:

```
C:\Program Files\MongoDB\Server\8.3\bin
```

 Look for:

- `mongod.exe`  (you already used this)
- `mongosh.exe` 

CASE 1: `mongosh.exe` is NOT there (most likely)

MongoDB 8.x often installs shell separately.

 Install MongoDB Shell

1. Go here:
<https://www.mongodb.com/try/download/shell>
2. Download **Windows (msi)**
3. Install it (default options)

Step 2: Run `mongosh` using full path

After install, try:

```
bat
```

Optional: Add to PATH (so you can just type `mongosh`)

1. Search: **Environment Variables**
2. Edit **Path**
3. Add:

```
C:\Program Files\MongoDB\mongosh\bin
```

Then reopen CMD and run:

bat



After this works

Run:

```
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "127.0.0.1:27017" },
    { _id: 1, host: "127.0.0.1:27018" },
    { _id: 2, host: "127.0.0.1:27019" }
  ]
})
```



Quick sanity check

If this command works:

bat



You're ready.

If you're unsure whether `mongosh` installed correctly, tell me what files you see in:

C:\Program Files\MongoDB\

I'll guide you exactly 👍



that mongosh have some files in zip it doesnt install when right click

Yeah—that's normal. The MongoDB Shell download you got is a **ZIP version**, not an installer—so right-click won't show "Install".

You just need to **extract and run it manually** 📌

✅ STEP-BY-STEP (ZIP version)

📦 Step 1: Extract the ZIP

1. Right-click the `.zip` file
2. Click "Extract All..."
3. Extract to something simple like:

`C:\mongosh`

📁 Step 2: Open the folder

Go inside:

`C:\mongosh`

You should see something like:

- `mongosh.exe` ✅

▶ Step 3: Run it

Open Command Prompt and run:

bat

👉 If everything is correct, you'll see:

```
test>
```

🔗 STEP 4: Initialize your cluster (important)

Once inside shell, run:

```
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "127.0.0.1:27017" },
    { _id: 1, host: "127.0.0.1:27018" },
    { _id: 2, host: "127.0.0.1:27019" }
  ]
})
```

⚡ Optional (make life easier)

Add it to PATH so you can just type `mongosh` :

1. Search → Environment Variables
2. Edit Path
3. Add:

```
C:\mongosh
```

Then you can run:

```
bat
```



Quick check

If this works:

```
bat
```

👉 You're good.

🚀 You're almost done

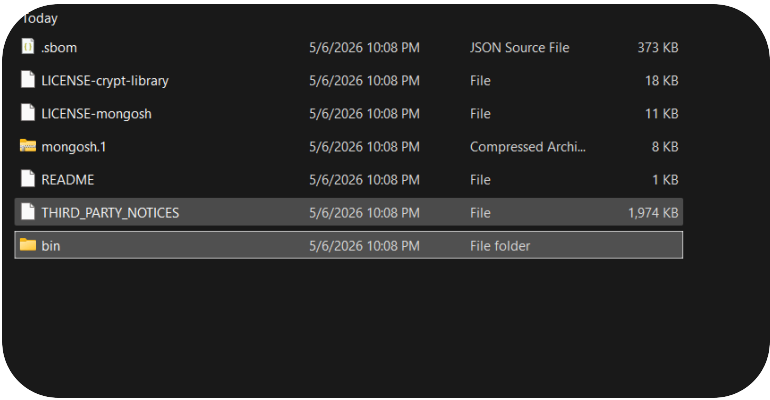
Once you run `rs.initiate()` :

- That previous `oplog.rs not found` error will disappear
- Your cluster becomes active

If anything fails, send:

- Screenshot or error
- Your folder structure

I'll debug it with you step-by-step 👉



this are files

Perfect—that screenshot actually shows everything is **correctly extracted** 👉

The key thing you missed is this:

 mongosh.exe is inside the bin folder

What to do now

Step 1: Open the bin folder

Double-click:

bin

Inside you should see:

mongosh.exe 

Step 2: Run mongosh

Now do one of these:

Easiest way (recommended)

1. Click the address bar in that folder
2. Type:

bat

3. Press Enter → Command Prompt opens in that folder

Then run:

bat

OR run from anywhere

If your folder is like:

C:\Users\YourName\Downloads\mongosh\bin

Then run:

bat

Step 3: Initialize your cluster

Once you see something like:

test>

Run this:

```
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "127.0.0.1:27017" },
    { _id: 1, host: "127.0.0.1:27018" },
    { _id: 2, host: "127.0.0.1:27019" }
  ]
})
```

What will happen next

- Your earlier error (oplog.rs not found) →  disappears
- Replica set becomes active
- Cluster starts working

Quick summary

From your screenshot:

-  Files are correct

-  You extracted properly
-  Just need to go into `bin` folder

If you want, next I can:

- Help you confirm PRIMARY/SECONDARY
- Or show a simple test to prove cluster is working

Just tell me 



Please enter a MongoDB connection string (Default: `mongodb://localhost/`):

That prompt is totally normal  — it's from the MongoDB Shell asking where to connect.

What to type here

Since your cluster is running locally, just press **Enter** 

(Default: `mongodb://localhost/`)

 This connects to:

`mongodb://localhost:27017`

OR (better for replica set)

You can directly connect to your cluster like this:

Bash

👉 Paste that and press Enter



After connecting

You'll see something like:

```
test>
```

Now run:

```
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "127.0.0.1:27017" },
    { _id: 1, host: "127.0.0.1:27018" },
    { _id: 2, host: "127.0.0.1:27019" }
  ]
})
```



Then check

JavaScript



Simple rule

Situation

Prompt asks for connection string

Want full cluster connection

What to do

Press Enter ☒

Paste replica string

If you get any error after `rs.initiate()` , paste it here—I'll fix it with you 🍌



Please enter a MongoDB connection string (Default: `mongodb://localhost/`):

Current Mongosh Log ID: 69fb6f2101db5062fbabc113
 Connecting to: `mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3`
 Using MongoDB: 8.3.1
 Using Mongosh: 2.8.3

For mongosh info see:

<https://www.mongodb.com/docs/mongodb-shell/>

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (<https://www.mongodb.com/legal/privacy-policy>). You can opt-out by running the `disableTelemetry()` command.

The server generated these startup warnings when booting

2026-05-06T22:04:12.593+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted

test>

Perfect 🍌 — this means everything is working correctly so far.

You are now **successfully connected** to MongoDB and ready for the final step.



Now do THIS (very important)

Inside the shell where you see:

```
test>
```



Run this command:

```
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "127.0.0.1:27017" },
    { _id: 1, host: "127.0.0.1:27018" },
    { _id: 2, host: "127.0.0.1:27019" }
  ]
})
```



Wait a few seconds (5–10 sec)

Then run:

JavaScript



What you should see

- One node:

```
"stateStr" : "PRIMARY"
```

- Two nodes:

```
"stateStr" : "SECONDARY"
```




That warning you saw

Access control is not enabled



Ignore it for now — it just means:

- No username/password yet
- Totally fine for learning



After this step

- Your earlier error (`oplog.rs not found`) → ☒ gone
- Replica set → ☒ active
- Cluster → ☒ working



Quick test (after setup)

Run:

```
use testdb
```

JavaScript

Then connect to another node (port 27018) and check data.



You're at the final step

Just run `rs.initiate()` now.

If anything fails, paste the error—I'll fix it instantly



```
test> rs.initiate({
```

```
| _id: "rs0",
| members: [
|   { _id: 0, host: "127.0.0.1:27017" },
|   { _id: 1, host: "127.0.0.1:27018" },
|   { _id: 2, host: "127.0.0.1:27019" }
| ]
| })
```

MongoServerError[NodeNotFound]: replSetInitiate quorum check failed because not all proposed set members responded affirmatively: 127.0.0.1:27018 failed with Error connecting to 127.0.0.1:27018 :: caused by :: onInvoke :: caused by :: No connection could be made because the target machine actively refused it., 127.0.0.1:27019 failed with Error connecting to 127.0.0.1:27019 :: caused by :: onInvoke :: caused by :: No connection could be made because the target machine actively refused it.

test>

Good—this error is actually very clear and fixable 👍

! What your error means

No connection could be made because the target machine actively refused it

👉 MongoDB is trying to contact:

- 127.0.0.1:27018
- 127.0.0.1:27019

✗ But nothing is running on those ports

💬 Translation (simple)

Port

Status

27017

✅ Running

Port	Status
27018	✗ NOT running
27019	✗ NOT running

So the replica set **cannot** form quorum

✅ FIX (do this now)

● Step 1: Start node2

Open a NEW Command Prompt:

```
bat
```

● Step 2: Start node3

Open another Command Prompt:

```
bat
```

⚠ Important

Make sure:

- Both windows stay OPEN
- No errors appear

🔍 Step 3: Verify they are running

You should see logs like:

```
Waiting for connections on port 27018  
Waiting for connections on port 27019
```

Step 4: Run initiate AGAIN

Go back to your shell and run:

```
rs.initiate({  
  _id: "rs0",  
  members: [  
    { _id: 0, host: "127.0.0.1:27017" },  
    { _id: 1, host: "127.0.0.1:27018" },  
    { _id: 2, host: "127.0.0.1:27019" }  
  ]  
})
```

Expected result

```
{ "ok" : 1 }
```

Then:

JavaScript

If it STILL fails

Tell me:

- What appears in node2 window
- What appears in node3 window

Key lesson

Replica set needs:

👉 ALL nodes running before `rs.initiate()`

You're one step away now—start node2 + node3 and retry 👍



```
C:\Program Files\MongoDB\Server\8.3\bin> > mongod.exe  
--config C:\mongo-cluster\config\node2.cfg  
Access is denied.
```

```
C:\Program Files\MongoDB\Server\8.3\bin>
```

Good, this is a **Windows permission issue**, not a MongoDB problem 👍

! Why you got this

```
Access is denied.
```

👉 Common reasons:

- You're not running Command Prompt as **Administrator**
- Windows is blocking access to the folder (`C:\mongo-cluster`)
- Or antivirus/permissions issue

✅ FIX (do this exactly)

🟢 Step 1: Run Command Prompt as Administrator

1. Click **Start**
2. Search: `cmd`
3. Right-click → **Run as administrator**

👉 This is important

🟢 Step 2: Run the command again

```
bat
```

🟡 If it STILL says Access Denied

Then it's likely folder permission.

🔧 Fix folder permissions

Run this in admin CMD:

```
bat
```

👉 This gives full control (fine for learning)

🟡 Alternative quick fix (very reliable)

Move your cluster folder to your user directory:

```
bat
```

Then recreate config paths like:

```
YAML
```

👉 Windows is much less strict there



Also check this

Make sure:

- `node2.cfg` exists
- Path is correct
- No typo in file name



After fixing

Start both:

```
mongod --config node2.cfg
```

```
bat
```

Then go back and run:

```
JavaScript
```



Reality

This is the **last blocker** before your cluster works.

If it still fails, send:

- Screenshot of folder `C:\mongo-cluster`
- Your config file

I'll pinpoint the issue instantly 👍



Microsoft Windows [Version 10.0.26200.8328]
 (c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>cd "C:\Program
 Files\MongoDB\Server\8.3\bin"

```
C:\Program Files\MongoDB\Server\8.3\bin>mongod.exe --
config C:\mongo-cluster\config\node1.cfg
{"t":{"$date":"2026-05-06T22:14:39.735+05:30"},"s":"I",
"c":"-", "id":8991200, "ctx":"thread1", "msg":"Shuffling
initializers", "attr":{"seed":776443339}}
{"t":{"$date":"2026-05-06T22:14:39.752+05:30"},"s":"I",
"c":"CONTROL", "id":97374,
"ctx":"thread1", "msg":"Automatically disabling TLS 1.0 and
TLS 1.1, to force-enable TLS 1.1 specify --
sslDisabledProtocols 'TLS1_0'; to force-enable TLS 1.0
specify --sslDisabledProtocols 'none'"}
{"t":{"$date":"2026-05-06T22:14:40.017+05:30"},"s":"I",
"c":"EXECUTOR",
"id":11280000, "ctx":"thread1", "msg":"Starting thread
pool", "attr":
{"poolName":"AuthorizationRouter", "numThreads":0, "minT
hreads":0, "maxThreads":1000000000}}
{"t":{"$date":"2026-05-06T22:14:40.017+05:30"},"s":"I",
"c":"NETWORK", "id":4915701,
"ctx":"thread1", "msg":"Initialized wire specification", "attr":
{"spec":{"incomingExternalClient":
{"minWireVersion":0, "maxWireVersion":28}, "incomingIntern
alClient":
{"minWireVersion":0, "maxWireVersion":28}, "outgoing":
{"minWireVersion":6, "maxWireVersion":28}, "isInternalClient
":true}}}
{"t":{"$date":"2026-05-06T22:14:40.018+05:30"},"s":"I",
"c":"EXECUTOR",
"id":11280000, "ctx":"thread1", "msg":"Starting thread
pool", "attr":
{"poolName":"ThreadPool1", "numThreads":0, "minThreads":
0, "maxThreads":1}}
```



```

{"t":{"$date":"2026-05-06T22:14:40.019+05:30"},"s":"I",
"c":"CONTROL", "id":5945603, "ctx":"thread1","msg":"Multi
threading initialized"}
{"t":{"$date":"2026-05-06T22:14:40.019+05:30"},"s":"I",
"c":"EXECUTOR",
"id":11280000,"ctx":"thread1","msg":"Starting thread
pool","attr":
{"poolName":"ReadWriteConcernDefaults","numThreads":0
,"minThreads":0,"maxThreads":1}}
{"t":{"$date":"2026-05-06T22:14:40.019+05:30"},"s":"I",
"c":"CONTROL", "id":4615611,
"ctx":"initandlisten","msg":"MongoDB starting","attr":
{"pid":1452,"port":27017,"dbPath":"C:/mongo-
cluster/node1","architecture":"64-bit","host":"Mrudula"}}
{"t":{"$date":"2026-05-06T22:14:40.026+05:30"},"s":"I",
"c":"CONTROL", "id":23398,
"ctx":"initandlisten","msg":"Target operating system
minimum version","attr":{"targetMinOS":"Windows
7/Windows Server 2008 R2"}}
{"t":{"$date":"2026-05-06T22:14:40.027+05:30"},"s":"I",
"c":"CONTROL", "id":23403,
"ctx":"initandlisten","msg":"Build Info","attr":{"buildInfo":
{"version":"8.3.1","gitVersion":"094e631246d5b5bee5bd5f2
0b12f882bc8e286ea","modules":[],"allocator":"tcmalloc-
gperf","environment":
{"distmod":"windows","distarch":"amd64","target_arch":"a
md64"}}}}
{"t":{"$date":"2026-05-06T22:14:40.027+05:30"},"s":"I",
"c":"CONTROL", "id":51765,
"ctx":"initandlisten","msg":"Operating System","attr":{"os":
{"name":"Microsoft Windows Workstation (build
26200)","version":"10.0 (build 26200)"}}
{"t":{"$date":"2026-05-06T22:14:40.027+05:30"},"s":"I",
"c":"CONTROL", "id":21951,
"ctx":"initandlisten","msg":"Options set by command
line","attr":{"options":{"config":"C:\\mongo-
cluster\\config\\node1.cfg","net":
{"bindIp":"127.0.0.1","port":27017},"replication":
{"replSetName":"rs0"},"storage":{"dbPath":"C:\\mongo-
cluster\\node1"}}}}

```

```

{"t":{"$date":"2026-05-06T22:14:40.027+05:30"},"s":"I",
"c":"EXECUTOR",
"id":11280000,"ctx":"initandlisten","msg":"Starting thread
pool","attr":
{"poolName":"OCSPManagerHTTP","numThreads":0,"minT
hreads":1,"maxThreads":10}}
{"t":{"$date":"2026-05-06T22:14:40.028+05:30"},"s":"W",
"c":"NETWORK",
"id":11621101,"ctx":"initandlisten","msg":"Overriding max
connections to honor
  capMemoryConsumptionForPreAuthBuffers  settings","attr":
{"limit":205508}}
{"t":{"$date":"2026-05-06T22:14:40.029+05:30"},"s":"I",
"c":"NETWORK", "id":4648601,
"ctx":"initandlisten","msg":"Implicit TCP FastOpen
unavailable. If TCP FastOpen is required, set at least one of
the related parameters","attr":{"relatedParameters":
["tcpFastOpenServer","tcpFastOpenClient","tcpFastOpenQu
eueSize"]}}
{"t":{"$date":"2026-05-06T22:14:40.030+05:30"},"s":"I",
"c":"STORAGE", "id":22270,
"ctx":"initandlisten","msg":"Storage engine to use detected
by data files","attr":{"dbpath":"C:/mongo-
cluster/node1","storageEngine":"wiredTiger"}}
{"t":{"$date":"2026-05-06T22:14:40.031+05:30"},"s":"I",
"c":"STORAGE", "id":22315,
"ctx":"initandlisten","msg":"Opening WiredTiger","attr":
{"config":"create,cache_size=7515M,session_max=33000,ev
iction=
(threads_min=4,threads_max=4),config_base=false,statistic
s=(fast),log=
(enabled=true,remove=true,path=journal,compressor=sna
ppy),builtin_extension_config=(zstd=
(compression_level=6)),file_manager=
(close_idle_time=600,close_scan_interval=10,close_handle_
minimum=2000),statistics_log=(wait=0),json_output=
(error,message),verbose=
[recovery_progress:1,checkpoint_progress:1,compact_progr
ess:1,live_restore_progress:1,backup:0,checkpoint:0,compa
ct:0,disaggregated_storage:0,eviction:0,fileops:0,history_sto

```

```

re:0,live_restore:0,recovery:0,rts:0,salvage:0,tiered:0,timesta
mp:0,transaction:0,verify:0,log:0],prefetch=
(available=true,default=false),"}
{"t":{"$date":"2026-05-06T22:14:40.034+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":32893,"thread":"1452:14072
8292836896","session_name":"wiredtiger_open","category"
:"WT_VERB_RECOVERY","log_id":1000000,"category_id":34,"
verbose_level":"INFO","verbose_level_id":0,"msg":"opening
the WiredTiger library"}}}
{"t":{"$date":"2026-05-06T22:14:40.051+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":51139,"thread":"1452:14072
8292836896","session_name":"connection","category":"WT
_VERB_RECOVERY","log_id":1000000,"category_id":34,"verb
ose_level":"INFO","verbose_level_id":0,"msg":"connection
configuration string parsing completed"}}}
{"t":{"$date":"2026-05-06T22:14:40.054+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":54134,"thread":"1452:14072
8292836896","session_name":"connection","category":"WT
_VERB_RECOVERY","log_id":1000000,"category_id":34,"verb
ose_level":"INFO","verbose_level_id":0,"msg":"starting
WiredTiger utility threads"}}}
{"t":{"$date":"2026-05-06T22:14:40.060+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":59549,"thread":"1452:14072
8292836896","session_name":"connection","category":"WT
_VERB_RECOVERY","log_id":1000000,"category_id":34,"verb
ose_level":"INFO","verbose_level_id":0,"msg":"starting
WiredTiger recovery"}}}
{"t":{"$date":"2026-05-06T22:14:40.062+05:30"},"s":"I",

```

```

"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":61544,"thread":"1452:14072
8292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY_PROGRESS","log
_id":1000000,"category_id":35,"verbose_level":"DEBUG_1","
verbose_level_id":1,"msg":"Recovering log 2 through 3"}}
{"t":{"$date":"2026-05-06T22:14:40.120+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":119838,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY_PROGRESS","log
_id":1000000,"category_id":35,"verbose_level":"DEBUG_1","
verbose_level_id":1,"msg":"Recovering log 3 through 3"}}
{"t":{"$date":"2026-05-06T22:14:40.175+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":174599,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY","log_id":100000
0,"category_id":34,"verbose_level":"INFO","verbose_level_id
":0,"msg":"scanning metadata to remove all incomplete
tables"}}}
{"t":{"$date":"2026-05-06T22:14:40.175+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":174599,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY","log_id":100000
0,"category_id":34,"verbose_level":"INFO","verbose_level_id
":0,"msg":"scanning metadata to find the largest file ID"}}
{"t":{"$date":"2026-05-06T22:14:40.177+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":

```

```

{"ts_sec":1778085880,"ts_usec":176936,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY","log_id":100000
0,"category_id":34,"verbose_level":"INFO","verbose_level_id
":0,"msg":"largest file ID found in the metadata 120"}}
{"t":{"$date":"2026-05-06T22:14:40.179+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":178402,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY","log_id":100000
0,"category_id":34,"verbose_level":"INFO","verbose_level_id
":0,"msg":"Main recovery loop: starting at 2,6912 to
3,256"}}}
{"t":{"$date":"2026-05-06T22:14:40.275+05:30"},"s":"I",
"c":"WTEVICT", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":275115,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_EVICTION","log_id":100000
0,"category_id":15,"verbose_level":"INFO","verbose_level_id
":0,"msg":"starting eviction threads"}}}
{"t":{"$date":"2026-05-06T22:14:40.276+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":275115,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY_PROGRESS","log
_id":1000000,"category_id":35,"verbose_level":"DEBUG_1","
verbose_level_id":1,"msg":"Recovering log 2 through 3"}}}
{"t":{"$date":"2026-05-06T22:14:40.354+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":354823,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY_PROGRESS","log

```

```

_id":1000000,"category_id":35,"verbose_level":"DEBUG_1","
verbose_level_id":1,"msg":"Recovering log 3 through 3"}}
{"t":{"$date":"2026-05-06T22:14:40.405+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":405231,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY","log_id":100000
0,"category_id":34,"verbose_level":"INFO","verbose_level_id
":0,"msg":"recovery log replay has successfully finished and
ran for 344 milliseconds"}}}
{"t":{"$date":"2026-05-06T22:14:40.406+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":405231,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY_PROGRESS","log
_id":1000000,"category_id":35,"verbose_level":"DEBUG_1","
verbose_level_id":1,"msg":"Set global recovery timestamp:
(0, 0)"}}}}
{"t":{"$date":"2026-05-06T22:14:40.407+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":407236,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY_PROGRESS","log
_id":1000000,"category_id":35,"verbose_level":"DEBUG_1","
verbose_level_id":1,"msg":"Set global oldest timestamp: (0,
0)"}}}}
{"t":{"$date":"2026-05-06T22:14:40.408+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":407236,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY","log_id":100000
0,"category_id":34,"verbose_level":"INFO","verbose_level_id

```

```

":0,"msg":"[RECOVERY_RTS] performing recovery
rollback_to_stable with stable_timestamp=(0, 0) and
oldest_timestamp=(0, 0)"}
{"t":{"$date":"2026-05-06T22:14:40.410+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":410237,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY","log_id":100000
0,"category_id":34,"verbose_level":"INFO","verbose_level_id
":0,"msg":"recovery rollback to stable has successfully
finished and ran for 3 milliseconds"}}}
{"t":{"$date":"2026-05-06T22:14:40.411+05:30"},"s":"I",
"c":"WTEVICT", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":410237,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_EVICTION","log_id":100000
0,"category_id":15,"verbose_level":"INFO","verbose_level_id
":0,"msg":"stopping eviction threads"}}}
{"t":{"$date":"2026-05-06T22:14:40.411+05:30"},"s":"I",
"c":"WTEVICT", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":411539,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_EVICTION","log_id":100000
0,"category_id":15,"verbose_level":"INFO","verbose_level_id
":0,"msg":"waiting for eviction threads to stop"}}}
{"t":{"$date":"2026-05-06T22:14:40.413+05:30"},"s":"I",
"c":"WTEVICT", "id":22430,
"ctx":"thread7","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":413549,"thread":"1452:1407
28292836896","session_dhandle_name":"file:WiredTigerHS.
wt","session_name":"eviction-
server","category":"WT_VERB_EVICTION","log_id":1000000,
"category_id":15,"verbose_level":"INFO","verbose_level_id":

```

```

0,"msg":"eviction thread exiting"}}}
{"t":{"$date":"2026-05-06T22:14:40.415+05:30"},"s":"I",
"c":"WTCHKPT", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":414545,"thread":"1452:1407
28292836896","session_name":"WT_SESSION.checkpoint","
category":"WT_VERB_CHECKPOINT","log_id":1000000,"cate
gory_id":5,"verbose_level":"INFO","verbose_level_id":0,"msg
":"Checkpoint requested at stable timestamp (0, 0)}}}
{"t":{"$date":"2026-05-06T22:14:40.416+05:30"},"s":"I",
"c":"WTCHKPT", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":415825,"thread":"1452:1407
28292836896","session_name":"WT_SESSION.checkpoint","
category":"WT_VERB_CHECKPOINT_PROGRESS","log_id":10
00000,"category_id":7,"verbose_level":"INFO","verbose_lev
el_id":0,"msg":"saving checkpoint snapshot min: 1, snapshot
max: 1 snapshot count: 0, oldest timestamp: (0, 0) , meta
checkpoint timestamp: (0, 0) base write gen: 39"}}}
{"t":{"$date":"2026-05-06T22:14:40.421+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":420694,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY","log_id":100000
0,"category_id":34,"verbose_level":"INFO","verbose_level_id
":0,"msg":"recovery checkpoint has successfully finished
and ran for 7 milliseconds"}}}
{"t":{"$date":"2026-05-06T22:14:40.422+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":421785,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY","log_id":149320
1,"category_id":34,"verbose_level":"INFO","verbose_level_id
":0,"msg":"recovery was completed successfully and took

```



```

361ms, including 344ms for the log replay, 3ms for the
rollback to stable, and 7ms for the checkpoint.}}}
{"t":{"$date":"2026-05-06T22:14:40.422+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":421785,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY_PROGRESS","log
_id":1493201,"category_id":35,"verbose_level":"INFO","verb
ose_level_id":0,"msg":"recovery was completed successfully
and took 361ms, including 344ms for the log replay, 3ms
for the rollback to stable, and 7ms for the checkpoint.}}}
{"t":{"$date":"2026-05-06T22:14:40.424+05:30"},"s":"I",
"c":"WTEVICT", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":423211,"thread":"1452:1407
28292836896","session_name":"connection","category":"W
T_VERB_EVICTION","log_id":1000000,"category_id":15,"verb
ose_level":"INFO","verbose_level_id":0,"msg":"starting
eviction threads"}}}
{"t":{"$date":"2026-05-06T22:14:40.426+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":425478,"thread":"1452:1407
28292836896","session_name":"connection","category":"W
T_VERB_RECOVERY","log_id":1000000,"category_id":34,"ver
bose_level":"INFO","verbose_level_id":0,"msg":"WiredTiger
utility threads started successfully"}}}
{"t":{"$date":"2026-05-06T22:14:40.426+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":425478,"thread":"1452:1407
28292836896","session_name":"connection","category":"W
T_VERB_RECOVERY","log_id":1000000,"category_id":34,"ver
bose_level":"INFO","verbose_level_id":0,"msg":"the
WiredTiger library has successfully opened"}}}

```

```

{"t":{"$date":"2026-05-06T22:14:40.427+05:30"},"s":"I",
"c":"STORAGE", "id":4795906,
"ctx":"initandlisten","msg":"WiredTiger opened","attr":
{"durationMillis":396}}
{"t":{"$date":"2026-05-06T22:14:40.430+05:30"},"s":"I",
"c":"RECOVERY", "id":23987,
"ctx":"initandlisten","msg":"WiredTiger
recoveryTimestamp","attr":{"recoveryTimestamp":
{"$timestamp":{"t":0,"i":0}}}}
{"t":{"$date":"2026-05-06T22:14:40.430+05:30"},"s":"I",
"c":"STORAGE", "id":9086700,
"ctx":"initandlisten","msg":"WiredTiger session cache max
value has been set","attr":{"sessionCacheMax":16500}}
{"t":{"$date":"2026-05-06T22:14:40.434+05:30"},"s":"I",
"c":"STORAGE",
"id":10158000,"ctx":"initandlisten","msg":"Opening spill
WiredTiger","attr":
{"config":"create,cache_size=401M,session_max=1024,evict
ion=
(threads_min=1,threads_max=1),eviction_dirty_target=20M
B,eviction_dirty_trigger=320MB,eviction_updates_trigger=
320MB,config_base=false,statistics=
(fast),builtin_extension_config=(zstd=
(compression_level=-7)),file_manager=
(close_idle_time=600,close_scan_interval=10,close_handle_
minimum=2000),statistics_log=(wait=0),json_output=
(error,message),verbose=
[recovery_progress:1,checkpoint_progress:1,compact_progr
ess:1,live_restore_progress:1,backup:0,checkpoint:0,compa
ct:0,disaggregated_storage:0,eviction:0,fileops:0,history_sto
re:0,live_restore:0,recovery:0,rts:0,salvage:0,tiered:0,timesta
mp:0,transaction:0,verify:0,log:0],"}}
{"t":{"$date":"2026-05-06T22:14:40.437+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":436593,"thread":"1452:1407
28292836896","session_name":"wiredtiger_open","category
":"WT_VERB_RECOVERY","log_id":1000000,"category_id":34,
"verbose_level":"INFO","verbose_level_id":0,"msg":"opening

```

```

the WiredTiger library}}
{"t":{"$date":"2026-05-06T22:14:40.439+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":439606,"thread":"1452:1407
28292836896","session_name":"connection","category":"W
T_VERB_RECOVERY","log_id":1000000,"category_id":34,"ver
bose_level":"INFO","verbose_level_id":0,"msg":"connection
configuration string parsing completed"}}}
{"t":{"$date":"2026-05-06T22:14:40.446+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":446438,"thread":"1452:1407
28292836896","session_name":"connection","category":"W
T_VERB_RECOVERY","log_id":1000000,"category_id":34,"ver
bose_level":"INFO","verbose_level_id":0,"msg":"starting
WiredTiger utility threads"}}}
{"t":{"$date":"2026-05-06T22:14:40.447+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":446438,"thread":"1452:1407
28292836896","session_name":"connection","category":"W
T_VERB_RECOVERY","log_id":1000000,"category_id":34,"ver
bose_level":"INFO","verbose_level_id":0,"msg":"starting
WiredTiger recovery"}}}
{"t":{"$date":"2026-05-06T22:14:40.448+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":447932,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY","log_id":100000
0,"category_id":34,"verbose_level":"INFO","verbose_level_id
":0,"msg":"scanning metadata to remove all incomplete
tables"}}}
{"t":{"$date":"2026-05-06T22:14:40.448+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,

```

```

"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":447932,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY","log_id":100000
0,"category_id":34,"verbose_level":"INFO","verbose_level_id
":0,"msg":"scanning metadata to find the largest file ID"}}
{"t":{"$date":"2026-05-06T22:14:40.448+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":447932,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY","log_id":100000
0,"category_id":34,"verbose_level":"INFO","verbose_level_id
":0,"msg":"largest file ID found in the metadata 0"}}
{"t":{"$date":"2026-05-06T22:14:40.448+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":447932,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY","log_id":100000
0,"category_id":34,"verbose_level":"INFO","verbose_level_id
":0,"msg":"recovery log replay has successfully finished and
ran for 1 milliseconds"}}
{"t":{"$date":"2026-05-06T22:14:40.449+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":448940,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY_PROGRESS","log
_id":1000000,"category_id":35,"verbose_level":"DEBUG_1",
"verbose_level_id":1,"msg":"Set global recovery timestamp:
(0, 0)"}
{"t":{"$date":"2026-05-06T22:14:40.449+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":

```

```

{"ts_sec":1778085880,"ts_usec":448940,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY_PROGRESS","log
_id":1000000,"category_id":35,"verbose_level":"DEBUG_1","
verbose_level_id":1,"msg":"Set global oldest timestamp: (0,
0)"}
{"t":{"$date":"2026-05-06T22:14:40.449+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":448940,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY","log_id":149320
1,"category_id":34,"verbose_level":"INFO","verbose_level_id
":0,"msg":"recovery was completed successfully and took
2ms, including 1ms for the log replay, 0ms for the rollback
to stable, and 0ms for the checkpoint."}}}
{"t":{"$date":"2026-05-06T22:14:40.449+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":449946,"thread":"1452:1407
28292836896","session_name":"txn-
recover","category":"WT_VERB_RECOVERY_PROGRESS","log
_id":1493201,"category_id":35,"verbose_level":"INFO","verb
ose_level_id":0,"msg":"recovery was completed successfully
and took 2ms, including 1ms for the log replay, 0ms for
the rollback to stable, and 0ms for the checkpoint."}}}
{"t":{"$date":"2026-05-06T22:14:40.460+05:30"},"s":"I",
"c":"WTEVICT", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":460105,"thread":"1452:1407
28292836896","session_name":"connection","category":"W
T_VERB_EVICTION","log_id":1000000,"category_id":15,"verb
ose_level":"INFO","verbose_level_id":0,"msg":"starting
eviction threads"}}}
{"t":{"$date":"2026-05-06T22:14:40.462+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":

```

```

{"message":
{"ts_sec":1778085880,"ts_usec":461102,"thread":"1452:1407
28292836896","session_name":"connection","category":"W
T_VERB_RECOVERY","log_id":1000000,"category_id":34,"ver
bose_level":"INFO","verbose_level_id":0,"msg":"WiredTiger
utility threads started successfully"}}
{"t":{"$date":"2026-05-06T22:14:40.462+05:30"},"s":"I",
"c":"WTRECOV", "id":22430,
"ctx":"initandlisten","msg":"WiredTiger message","attr":
{"message":
{"ts_sec":1778085880,"ts_usec":462104,"thread":"1452:1407
28292836896","session_name":"connection","category":"W
T_VERB_RECOVERY","log_id":1000000,"category_id":34,"ver
bose_level":"INFO","verbose_level_id":0,"msg":"the
WiredTiger library has successfully opened"}}
{"t":{"$date":"2026-05-06T22:14:40.462+05:30"},"s":"I",
"c":"STORAGE",
"id":10158001,"ctx":"initandlisten","msg":"Spill WiredTiger
opened","attr":{"durationMillis":28}}
{"t":{"$date":"2026-05-06T22:14:40.463+05:30"},"s":"I",
"c":"STORAGE",
"id":11503101,"ctx":"initandlisten","msg":"Loading MDB
catalog"}
{"t":{"$date":"2026-05-06T22:14:40.464+05:30"},"s":"I",
"c":"STORAGE", "id":9529901,
"ctx":"initandlisten","msg":"Initializing durable
catalog","attr":{"numRecords":6}}
{"t":{"$date":"2026-05-06T22:14:40.465+05:30"},"s":"I",
"c":"STORAGE", "id":9529902,
"ctx":"initandlisten","msg":"Retrieving all idents from
storage engine"}
{"t":{"$date":"2026-05-06T22:14:40.466+05:30"},"s":"I",
"c":"STORAGE", "id":9529903,
"ctx":"initandlisten","msg":"Initializing all collections in
durable catalog","attr":{"numEntries":6}}
{"t":{"$date":"2026-05-06T22:14:40.466+05:30"},"s":"I",
"c":"STORAGE",
"id":11503103,"ctx":"initandlisten","msg":"Initializing
collection catalog"}
{"t":{"$date":"2026-05-06T22:14:40.467+05:30"},"s":"W",

```

```

"c":"CONTROL", "id":22120,
"ctx":"initandlisten","msg":"Access control is not enabled
for the database. Read and write access to data and
configuration is unrestricted","tags":["startupWarnings"]}
{"t":{"$date":"2026-05-06T22:14:40.471+05:30"},"s":"I",
"c":"NETWORK", "id":4915702,
"ctx":"initandlisten","msg":"Updated wire
specification","attr":{"oldSpec":{"incomingExternalClient":
{"minWireVersion":0,"maxWireVersion":28},"incomingIntern
alClient":
{"minWireVersion":0,"maxWireVersion":28},"outgoing":
{"minWireVersion":6,"maxWireVersion":28},"isInternalClient
":true},"newSpec":{"incomingExternalClient":
{"minWireVersion":0,"maxWireVersion":28},"incomingIntern
alClient":
{"minWireVersion":28,"maxWireVersion":28},"outgoing":
{"minWireVersion":28,"maxWireVersion":28},"isInternalClien
t":true}}}
{"t":{"$date":"2026-05-06T22:14:40.471+05:30"},"s":"I",
"c":"REPL", "id":5853300,
"ctx":"initandlisten","msg":"current
featureCompatibilityVersion value","attr":
{"featureCompatibilityVersion":"8.3","context":"startup"}}
{"t":{"$date":"2026-05-06T22:14:40.472+05:30"},"s":"I",
"c":"STORAGE", "id":5071100,
"ctx":"initandlisten","msg":"Clearing temp directory"}
{"t":{"$date":"2026-05-06T22:14:40.473+05:30"},"s":"I",
"c":"STORAGE",
"i":10682200,"ctx":"initandlisten","msg":"Dropping spill
idents","attr":{"numIdents":0}}
{"t":{"$date":"2026-05-06T22:14:40.475+05:30"},"s":"I",
"c":"CONTROL", "id":20536,
"ctx":"initandlisten","msg":"Flow Control is enabled on this
deployment"}
{"t":{"$date":"2026-05-06T22:14:40.477+05:30"},"s":"I",
"c":"EXTENSION","id":11901200,"ctx":"initandlisten","msg":
"Loading extensions on non-linux platforms is not
supported - skipping loading."}
{"t":{"$date":"2026-05-06T22:14:40.816+05:30"},"s":"I",
"c":"FTDC", "id":20625,

```

```

"ctx":"initandlisten","msg":"Initializing full-time diagnostic
data capture","attr":{"dataDirectory":"C:/mongo-
cluster/node1/diagnostic.data"}}
{"t":{"$date":"2026-05-06T22:14:40.816+05:30"},"s":"I",
"c":"QUERY", "id":8972602,
"ctx":"initandlisten","msg":"Setting up MozJS engine."}
{"t":{"$date":"2026-05-06T22:14:40.817+05:30"},"s":"I",
"c":"EXECUTOR",
"id":11280000,"ctx":"initandlisten","msg":"Starting thread
pool","attr":{"poolName":"deferred writer
pool","numThreads":0,"minThreads":0,"maxThreads":1}}
{"t":{"$date":"2026-05-06T22:14:40.818+05:30"},"s":"I",
"c":"EXECUTOR",
"id":11280000,"ctx":"initandlisten","msg":"Starting thread
pool","attr":
{"poolName":"StandaloneThreadPool","numThreads":0,"mi
nThreads":1,"maxThreads":1000000000}}
{"t":{"$date":"2026-05-06T22:14:40.818+05:30"},"s":"I",
"c":"EXECUTOR",
"id":11280000,"ctx":"initandlisten","msg":"Starting thread
pool","attr":
{"poolName":"WaitForMajorityServiceReadThreadPool","nu
mThreads":0,"minThreads":0,"maxThreads":2}}
{"t":{"$date":"2026-05-06T22:14:40.818+05:30"},"s":"I",
"c":"EXECUTOR",
"id":11280000,"ctx":"initandlisten","msg":"Starting thread
pool","attr":
{"poolName":"WaitForMajorityServiceWriteThreadPool","n
umThreads":0,"minThreads":0,"maxThreads":2}}
{"t":{"$date":"2026-05-06T22:14:40.824+05:30"},"s":"I",
"c":"EXECUTOR",
"id":11280000,"ctx":"initandlisten","msg":"Starting thread
pool","attr":
{"poolName":"MirrorMaestro","numThreads":0,"minThread
s":1,"maxThreads":2}}
{"t":{"$date":"2026-05-06T22:14:40.824+05:30"},"s":"I",
"c":"REPL", "id":40440,
"ctx":"initandlisten","msg":"Starting the
TopologyVersionObserver"}
{"t":{"$date":"2026-05-06T22:14:40.825+05:30"},"s":"I",

```



```

"c":"REPL",    "id":40445,
"ctx":"TopologyVersionObserver","msg":"Started
TopologyVersionObserver"}
{"t":{"$date":"2026-05-06T22:14:40.825+05:30"},"s":"I",
"c":"COMMAND",
"id":10735900,"ctx":"initandlisten","msg":"Defering
computation of targeted mirroring host list since config is
uninitialized"}
{"t":{"$date":"2026-05-06T22:14:40.829+05:30"},"s":"I",
"c":"EXECUTOR",
"id":11280000,"ctx":"initandlisten","msg":"Starting thread
pool","attr":
{"poolName":"ReplNodeDbWorkerThreadPool","numThrea
ds":0,"minThreads":1,"maxThreads":1000000000}}
{"t":{"$date":"2026-05-06T22:14:40.829+05:30"},"s":"I",
"c":"REPL",    "id":6015317,
"ctx":"initandlisten","msg":"Setting new configuration
state","attr":
{"newState":"ConfigStartingUp","oldState":"ConfigPreStart"
}}
{"t":{"$date":"2026-05-06T22:14:40.830+05:30"},"s":"I",
"c":"EXECUTOR",
"id":11280000,"ctx":"initandlisten","msg":"Starting thread
pool","attr":
{"poolName":"ReplCoordThreadPool","numThreads":0,"min
Threads":1,"maxThreads":50}}
{"t":{"$date":"2026-05-06T22:14:40.830+05:30"},"s":"I",
"c":"REPL",    "id":6005300,
"ctx":"initandlisten","msg":"Starting up replica set aware
services"}
{"t":{"$date":"2026-05-06T22:14:40.831+05:30"},"s":"I",
"c":"-",    "id":4939300,"ctx":"monitoring-keys-for-
HMAC","msg":"Failed to refresh key cache","attr":
{"error":"ReadConcernMajorityNotAvailableYet: Read
concern majority reads are currently not
possible.","nextRefreshIntervalMillis":200}}
{"t":{"$date":"2026-05-06T22:14:40.832+05:30"},"s":"I",
"c":"EXECUTOR",
"id":11280000,"ctx":"initandlisten","msg":"Starting thread
pool","attr":

```

```

{"poolName":"QueryAnalysisWriterThreadPool","numThreads":0,"minThreads":0,"maxThreads":8}}
{"t":{"$date":"2026-05-06T22:14:40.832+05:30"},"s":"I",
"c":"EXECUTOR",
"id":11280000,"ctx":"initandlisten","msg":"Starting thread pool","attr":
{"poolName":"InternalTransactionsReapService","numThreads":0,"minThreads":0,"maxThreads":1}}
{"t":{"$date":"2026-05-06T22:14:40.832+05:30"},"s":"I",
"c":"REPL", "id":4280500,
"ctx":"initandlisten","msg":"Attempting to create internal replication collections"}
{"t":{"$date":"2026-05-06T22:14:40.835+05:30"},"s":"I",
"c":"REPL", "id":4280501,
"ctx":"initandlisten","msg":"Attempting to load local voted for document"}
{"t":{"$date":"2026-05-06T22:14:40.835+05:30"},"s":"I",
"c":"REPL", "id":21311, "ctx":"initandlisten","msg":"Did not find local initialized voted for document at startup"}
{"t":{"$date":"2026-05-06T22:14:40.838+05:30"},"s":"I",
"c":"REPL", "id":4280502,
"ctx":"initandlisten","msg":"Searching for local Rollback ID document"}
{"t":{"$date":"2026-05-06T22:14:40.840+05:30"},"s":"I",
"c":"REPL", "id":21529,
"ctx":"initandlisten","msg":"Initializing rollback ID","attr":{"rbid":1}}
{"t":{"$date":"2026-05-06T22:14:40.841+05:30"},"s":"I",
"c":"REPL", "id":21313, "ctx":"initandlisten","msg":"Did not find local replica set configuration document at startup","attr":{"error":
{"code":47,"codeName":"NoMatchingDocument","errmsg":"Did not find replica set configuration document in local.system.replset"}}}
{"t":{"$date":"2026-05-06T22:14:40.841+05:30"},"s":"I",
"c":"REPL", "id":6015317,
"ctx":"initandlisten","msg":"Setting new configuration state","attr":
{"newState":"ConfigUninitialized","oldState":"ConfigStartingUp"}}

```

```

{"t":{"$date":"2026-05-06T22:14:40.841+05:30"},"s":"I",
"c":"EXECUTOR",
"id":11280000,"ctx":"initandlisten","msg":"Starting thread
pool","attr":
{"poolName":"TTLMonitorMetadataRefresh","numThreads"
:0,"minThreads":0,"maxThreads":5}}
{"t":{"$date":"2026-05-06T22:14:40.842+05:30"},"s":"I",
"c":"STORAGE", "id":22262,
"ctx":"initandlisten","msg":"Timestamp monitor starting"}
{"t":{"$date":"2026-05-06T22:14:40.843+05:30"},"s":"I",
"c":"EXECUTOR",
"id":11280000,"ctx":"initandlisten","msg":"Starting thread
pool","attr":
{"poolName":"FLECrud","numThreads":0,"minThreads":1,"m
axThreads":1000000000}}
{"t":{"$date":"2026-05-06T22:14:40.843+05:30"},"s":"I",
"c":"STORAGE", "id":7333401,
"ctx":"initandlisten","msg":"Starting the
DiskSpaceMonitor"}
{"t":{"$date":"2026-05-06T22:14:40.847+05:30"},"s":"I",
"c":"CONTROL", "id":20710,
"ctx":"LogicalSessionCacheRefresh","msg":"Failed to refresh
session cache, will try again at the next refresh
interval","attr":{"error":"NotYetInitialized: Replication has
not yet been configured"}}
{"t":{"$date":"2026-05-06T22:14:40.847+05:30"},"s":"I",
"c":"CONTROL", "id":20712,
"ctx":"LogicalSessionCacheReap","msg":"Sessions
collection is not set up; waiting until next sessions reap
interval","attr":{"error":"NamespaceNotFound:
config.system.sessions does not exist"}}
{"t":{"$date":"2026-05-06T22:14:40.847+05:30"},"s":"I",
"c":"EXECUTOR",
"id":11280000,"ctx":"initandlisten","msg":"Starting thread
pool","attr":
{"poolName":"StatsCache","numThreads":0,"minThreads":0,
"maxThreads":2}}
{"t":{"$date":"2026-05-06T22:14:40.847+05:30"},"s":"W",
"c":"SHARDING", "id":7012500,
"ctx":"QueryAnalysisConfigurationsRefresher","msg":"Failed

```

to refresh query analysis configurations, will try again at the next interval", "attr": {"error": "PrimarySteppedDown: No primary exists currently"}}

```
{
  "t": {"$date": "2026-05-06T22:14:40.848+05:30"},
  "s": "I",
  "c": "NETWORK",
  "id": 23015,
  "ctx": "listener",
  "msg": "Listening on",
  "attr": {
    "address": "127.0.0.1:27017"
  }
}
```

```
{
  "t": {"$date": "2026-05-06T22:14:40.849+05:30"},
  "s": "I",
  "c": "NETWORK",
  "id": 23016,
  "ctx": "listener",
  "msg": "Waiting for connections",
  "attr": {
    "port": 27017,
    "ssl": "off"
  }
}
```

```
{
  "t": {"$date": "2026-05-06T22:14:40.849+05:30"},
  "s": "I",
  "c": "CONTROL",
  "id": 8423403,
  "ctx": "initandlisten",
  "msg": "mongod startup complete",
  "attr": {
    "Summary of time elapsed": {
      "Startup from clean shutdown?": true,
      "Statistics": {
        "createLockFileMillis": 0,
        "setUpPeriodicRunnerMillis": 0,
        "setUpOCSPMillis": 0,
        "setUpTransportLayerMillis": 1,
        "initSyncCrashRecoveryMillis": 0,
        "getStorageEngineMetadataMillis": 0,
        "validateMetadataMillis": 0,
        "createStorageEngineMillis": 436,
        "initializeFCVForIndexMillis": 3,
        "dropAbandonedIdsMillis": 0,
        "userAndRolesGraphMillis": 0,
        "waitForMajorityServiceMillis": 1,
        "clusterTimeKeysManagerMillis": 0,
        "startUpReplCoordMillis": 12,
        "recoverChangeStreamMillis": 0,
        "logStartupOptionsMillis": 0,
        "startUpTransportLayerMillis": 2,
        "initAndListenTotalMillis": 830
      }
    }
  }
}
```

```
{
  "t": {"$date": "2026-05-06T22:14:40.849+05:30"},
  "s": "I",
  "c": "STORAGE",
  "id": 5295000,
  "ctx": "OplogCapMaintainerThread-local.oplog.rs",
  "msg": "Oplog cap maintainer thread started",
  "attr": {
    "threadName": "OplogCapMaintainerThread-local.oplog.rs"
  }
}
```

```
{
  "t": {"$date": "2026-05-06T22:14:41.005+05:30"},
  "s": "W",
  "c": "QUERY",
  "id": 23799,
  "ctx": "ftdc",
  "msg": "Aggregate command executor error",
  "attr": {
    "error": {
      "code": 26,
      "codeName": "NamespaceNotFound",
      "errmsg": "Unable to retrieve storageStats in $collStats stage :: caused by :: Collection [local.oplog.rs] not found.",
      "stats": {},
      "cmd": {
        "aggregate": "oplog.rs",
        "cursor": {},
        "pipeline": [
          {
            "$collStats":

```

```

{"storageStats":
{"waitForLock":false,"numericOnly":true}}},{"db":"local"}}
{"t":{"$date":"2026-05-06T22:14:41.032+05:30"},"s":"I",
"c":"-", "id":4939300, "ctx":"monitoring-keys-for-
HMAC","msg":"Failed to refresh key cache","attr":
{"error":"ReadConcernMajorityNotAvailableYet: Read
concern majority reads are currently not
possible.","nextRefreshIntervalMillis":400}}
{"t":{"$date":"2026-05-06T22:14:41.432+05:30"},"s":"I",
"c":"-", "id":4939300, "ctx":"monitoring-keys-for-
HMAC","msg":"Failed to refresh key cache","attr":
{"error":"ReadConcernMajorityNotAvailableYet: Read
concern majority reads are currently not
possible.","nextRefreshIntervalMillis":600}}
{"t":{"$date":"2026-05-06T22:14:42.003+05:30"},"s":"W",
"c":"QUERY", "id":23799, "ctx":"ftdc","msg":"Aggregate
command executor error","attr":{"error":
{"code":26,"codeName":"NamespaceNotFound","errmsg":
"Unable to retrieve storageStats in $collStats stage :: caused
by :: Collection [local.oplog.rs] not found."},"stats":{},"cmd":
{"aggregate":"oplog.rs","cursor":{},"pipeline":[{"$collStats":
{"storageStats":
{"waitForLock":false,"numericOnly":true}}},{"db":"local"}}
{"t":{"$date":"2026-05-06T22:14:42.033+05:30"},"s":"I",
"c":"-", "id":4939300, "ctx":"monitoring-keys-for-
HMAC","msg":"Failed to refresh key cache","attr":
{"error":"ReadConcernMajorityNotAvailableYet: Read
concern majority reads are currently not
possible.","nextRefreshIntervalMillis":800}}
{"t":{"$date":"2026-05-06T22:14:42.833+05:30"},"s":"I",
"c":"-", "id":4939300, "ctx":"monitoring-keys-for-
HMAC","msg":"Failed to refresh key cache","attr":
{"error":"ReadConcernMajorityNotAvailableYet: Read
concern majority reads are currently not
possible.","nextRefreshIntervalMillis":1000}}
{"t":{"$date":"2026-05-06T22:14:43.004+05:30"},"s":"W",
"c":"QUERY", "id":23799, "ctx":"ftdc","msg":"Aggregate
command executor error","attr":{"error":
{"code":26,"codeName":"NamespaceNotFound","errmsg":
"Unable to retrieve storageStats in $collStats stage :: caused

```

```

by :: Collection [local.oplog.rs] not found.},"stats":{},"cmd":
{"aggregate":"oplog.rs","cursor":{},"pipeline":[{"$collStats":
{"storageStats":
{"waitForLock":false,"numericOnly":true}}]},"$db":"local"}}
{"t":{"$date":"2026-05-06T22:14:43.834+05:30"},"s":"I",
"c":"-", "id":4939300, "ctx":"monitoring-keys-for-
HMAC","msg":"Failed to refresh key cache","attr":
{"error":"ReadConcernMajorityNotAvailableYet: Read
concern majority reads are currently not
possible.","nextRefreshIntervalMillis":1200}}
{"t":{"$date":"2026-05-06T22:14:44.002+05:30"},"s":"W",
"c":"QUERY", "id":23799, "ctx":"ftdc","msg":"Aggregate
command executor error","attr":{"error":
{"code":26,"codeName":"NamespaceNotFound","errmsg":
"Unable to retrieve storageStats in $collStats stage :: caused
by :: Collection [local.oplog.rs] not found.},"stats":{},"cmd":
{"aggregate":"oplog.rs","cursor":{},"pipeline":[{"$collStats":
{"storageStats":
{"waitForLock":false,"numericOnly":true}}]},"$db":"local"}}

```

This output is actually **perfect** 👍 — nothing is wrong here.

Let me explain clearly so you don't get confused.

✅ What just happened

Your MongoDB node **started successfully**:

```

Listening on 127.0.0.1:27017
mongod startup complete

```

👉 That means:

- Node1 (port 27017) = ✅ running

⚠️ About those warnings

You're seeing things like:

```
NamespaceNotFound: local.oplog.rs not found
Replication has not yet been configured
```

👉 These are **EXPECTED** right now

Why?

Because:

- You **haven't initialized the replica set yet**
- So MongoDB hasn't created replication data (oplog)

👉 These errors will disappear after `rs.initiate()`



What you still need to do

Right now you only started:

- ☒ node1 (27017)

But you still need:

- ☒ node2 (27018)
- ☒ node3 (27019)



NEXT STEPS (do this exactly)



Step 1: Open NEW terminal (Admin)

Start node2:

```
bat
```



Step 2: Open another NEW terminal

Start node3:

bat

Step 3: Confirm

You should see:

```
Waiting for connections on port 27018
Waiting for connections on port 27019
```

Step 4: Go back to mongosh

Run AGAIN:

```
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "127.0.0.1:27017" },
    { _id: 1, host: "127.0.0.1:27018" },
    { _id: 2, host: "127.0.0.1:27019" }
  ]
})
```

Expected result

```
{ ok: 1 }
```

Then:

JavaScript



Key idea (very important)

State

Current logs

After rs.initiate()

Meaning

Normal before replica set

Errors disappear



You are 90% done

Right now:

- MongoDB installed ✓
- Node1 running ✓
- Config correct ✓



Only missing:

Start node2 + node3

If node2/node3 fail again, paste that error—I'll fix it immediately 🙌



```
C:\Program Files\MongoDB\Server\8.3\bin> > mongod.exe  
--config C:\mongo-cluster\config\node2.cfg  
The process cannot access the file because it is being used  
by another process.
```

```
C:\Program Files\MongoDB\Server\8.3\bin>
```

Good — this is a **different issue now**, and it's very fixable 🙌



Error meaning

The process cannot access the file because it is being used by another process.

👉 This means:

- Either **node2** is already running, OR
- Another process is using **the same dbPath** or port

🔍 Most likely cause (very common)

👉 You probably:

- Started node2 earlier (even if it failed)
- Or node1/node2 are using **same folder** or port

✅ Step-by-step fix

🟢 Step 1: Check running MongoDB processes

Run:

```
bat
```

👉 If you see multiple processes like:

```
mongod.exe    1234
mongod.exe    5678
```

That means something is already running.

🟢 Step 2: Kill all MongoDB processes (clean reset)

```
bat
```

👉 This stops EVERYTHING (safe for now)

🟢 Step 3: Start nodes cleanly (VERY IMPORTANT ORDER)

Terminal 1:

```
bat
```

Terminal 2:

```
bat
```

Terminal 3:

```
bat
```

⚠️ DOUBLE-CHECK THIS (critical)

Open your config files and confirm:

node1.cfg

```
port: 27017
```

```
YAML
```

node2.cfg

```
port: 27018
```

```
YAML
```

node3.cfg

```
port: 27019
```

YAML

👉 If dbPath is same for multiple nodes, this error happens

🧠 Why this happens

MongoDB locks files inside dbPath.

👉 So:

- 2 processes **cannot use same folder**
- Windows throws "file in use"

🚀 After starting all 3

Go to shell:

```
bat
```

Then:

```
JavaScript
```

🔥 Quick summary

Problem

Fix

File in use

Kill mongod processes

Same dbPath

Use different folders

Same port

Use 27017/18/19

If you're unsure, paste your `node2.cfg` here — I'll verify it in seconds 👍



```
C:\Program Files\MongoDB\Server\8.3\bin>tasklist | findstr
mongod
mongod.exe           1452 Console          1
165,704 K
```

```
C:\Program Files\MongoDB\Server\8.3\bin>
```

Perfect — this output tells us exactly what's going on 👍

```
mongod.exe    1452
```

👉 Only **ONE** MongoDB process is running (your node1 on port 27017)

! So why node2 failed?

This error:

```
The process cannot access the file because it is being used by another process.
```

👉 **NOT** because of another mongod process

👉 It's almost certainly because **node2** is trying to use the **SAME** dbPath as node1

Let's confirm (very important)

Open this file:

```
C:\mongo-cluster\config\node2.cfg
```

Check this line:

```
YAML
```

 If you see this:

```
YAML
```

 That's the problem

Correct configuration (must be different)

node1.cfg

```
port: 27017
```

```
YAML
```

node2.cfg

```
port: 27018
```

```
YAML
```

node3.cfg

```
port: 27019
```

```
YAML
```

Step-by-step fix

1. Create missing folders (if needed)

```
mkdir C:\mongo-cluster\node2
```

```
bat
```

2. Fix node2.cfg (IMPORTANT)

Make sure it looks like:

```
systemLog:
  destination: file
  path: C:\mongo-cluster\node2.log
```

```
storage:
  dbPath: C:\mongo-cluster\node2
```

```
net:
  bindIp: 127.0.0.1
  port: 27018
```

```
replication:
  replSetName: rs0
```

3. Now start node2 again

```
bat
```

Key idea (this is the root cause)

 MongoDB locks the database folder

So:

- node1 → uses node1 folder
- node2 → MUST use node2 folder
- node3 → MUST use node3 folder

After this

Start node3 → then run:

JavaScript

If you want, paste your node2.cfg here — I'll double-check it line by line 

     ...

storage:

dbPath: C:\mongo-cluster\node2

net:

bindIp: 127.0.0.1

port: 27018

replication:

replSetName: rs0

Your config looks **correct** 👍 — nothing wrong with `node2.cfg` .

So if you're still getting:

```
The process cannot access the file because it is being used by another process.
```

then the issue is **not the config syntax** — it's one of these:

Most likely causes (in your case)

1. Folder already locked / reused

Even if only one `mongod` is running, Windows may still have a lock if:

- `node2` folder was used before
- or contains leftover lock files

FIX (do this exactly)

Step 1: Stop everything (clean reset)

Run:

```
bat
```

Step 2: Delete lock files (VERY IMPORTANT)

Go to:

```
C:\mongo-cluster\node2
```

👉 Delete this file if exists:

```
mongod.lock
```

Also safe to delete:

```
WiredTiger.lock
```

Step 3: (Optional but safest)

If you're just learning, you can **clear the whole folder**:

```
rmdir /s /q C:\mongo-cluster\node2
```

```
bat
```

Step 4: Start nodes again (fresh)

Terminal 1:

```
bat
```

Terminal 2:

```
bat
```

Terminal 3:

```
bat
```



One more thing to check

Make sure your `node2.log` is not locked:

If your config has:

systemLog:

YAML

👉 Ensure:

- Not open in Notepad
- Not used by another process

Why this happens

MongoDB creates lock files like:

- mongod.lock
- WiredTiger.lock

👉 If MongoDB crashes or is killed:

Windows may still think it's in use

After it works

Then run:

```
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "127.0.0.1:27017" },
    { _id: 1, host: "127.0.0.1:27018" },
    { _id: 2, host: "127.0.0.1:27019" }
  ]
})
```

Summary

Problem

File in use

Fix

Kill mongod

Problem

Lock file exists

Folder reused

Fix

Delete it

Reset folder

If it still fails, send:

- Full node2.cfg
- Screenshot of C:\mongo-cluster\node2

I'll pinpoint it instantly 🙌



```
C:\Windows\System32>>cd "C:\Program
Files\MongoDB\Server\8.3\bin"
"C:\Program Files\MongoDB\Server\8.3\bin" is not
recognized as an internal or external command,
operable program or batch file.
```

```
C:\Windows\System32>>cd "C:\Program
Files\MongoDB\Server\8.3\bin"
"C:\Program Files\MongoDB\Server\8.3\bin" is not
recognized as an internal or external command,
operable program or batch file.
```

```
C:\Windows\System32>
```

That error is just a small command syntax issue 🙌 — easy fix.

! Why it failed

```
"C:\Program Files\MongoDB\Server\8.3\bin" is not recognized
```

👉 This happens when:

- You accidentally typed **extra quotes** or wrong characters
- Or copied the command incorrectly

✅ Correct way to change directory

Type this **exactly**:

```
bat
```

🧠 Important detail

Why `/d` ?

- Without `/d` , `cd` **won't switch drives properly**
- It's best practice on Windows

✅ After that

You should see:

```
C:\Program Files\MongoDB\Server\8.3\bin>
```

🚀 Then run your node

```
bat
```

⚠️ If it STILL fails

Skip `cd` completely and run directly:

```
bat
```

👉 This avoids all path issues

Summary

Problem

`cd` not working

path issues

Fix

use `cd /d`

run full path command

You're very close now — once node2 and node3 start, your cluster will finally initialize 🚀

If anything errors next, paste it 👍



3 nodes running tell next steps

Nice — this is the fun part now. If all 3 nodes are running, you're ready to **form the cluster (replica set)** 🚀

Step 1: Connect to node1

Open a new terminal:

```
bat
```

You should see:

```
test>
```

✅ Step 2: Initialize the replica set

Run this EXACT command:

```
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "127.0.0.1:27017" },
    { _id: 1, host: "127.0.0.1:27018" },
    { _id: 2, host: "127.0.0.1:27019" }
  ]
})
```

✅ Step 3: Verify

Wait ~5–10 seconds, then run:

JavaScript

🎯 Expected result

You should see something like:

- One node:

```
"stateStr" : "PRIMARY"
```

- Two nodes:

```
"stateStr" : "SECONDARY"
```

👉 That means your cluster is working 🎉

✅ Step 4: Test replication

Run this on PRIMARY:

```
use testdb
```

JavaScript

Now check from another node

Open another terminal:

```
bat
```

Then:

```
rs.slaveOk()
```

```
use testdb
```

JavaScript

👉 You should see the same data ✅

⚠️ If rs.initiate fails

Common issues:

- ❌ ports wrong
- ❌ node not running
- ❌ firewall blocking

What you just built

You created a MongoDB Replica Set, which gives:

- ✓ High availability
- ✓ Automatic failover
- ✓ Data replication

Next things you can try

- Stop PRIMARY → see failover
- Add authentication (secure cluster)
- Try connecting using replica set connection string

If `rs.status()` looks weird or errors, paste it here — I'll debug it instantly 🙌



```
C:\Windows\System32>cd /d "C:\Program  
Files\MongoDB\Server\8.3\bin"
```

```
C:\Program Files\MongoDB\Server\8.3\bin>mongosh --  
port 27017  
'mongosh' is not recognized as an internal or external  
command,  
operable program or batch file.
```

```
C:\Program Files\MongoDB\Server\8.3\bin>
```

You're very close — this is just a **mongosh path issue**, not a cluster problem 🙌

Why this happened

'mongosh' is not recognized

👉 Means:

- `mongosh.exe` is **not** installed in this MongoDB folder
- Or it's not in **PATH**

👉 This is normal in newer versions of MongoDB — shell is separate.

✅ You already downloaded mongosh (zip)

From your earlier screenshot, you have something like:

```
mongosh-2.x.x-win32-x64\
```

✅ Step 1: Go to mongosh folder

Example:

```
bat
```

👉 Adjust path based on where you extracted it

✅ Step 2: Run mongosh directly

```
bat
```

✅ Step 3 (optional but BEST): Add mongosh to PATH

So you don't have to go to that folder every time:

Do this:

1. Copy path:

C:\Users\YourName\Downloads\mongosh-2.8.3-win32-x64\bin

2. Search: Environment Variables

3. Edit Path

4. Add new entry

5. Paste path

6. OK

Now you can run:

bat

from anywhere 



After mongosh opens

Run:

```
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "127.0.0.1:27017" },
    { _id: 1, host: "127.0.0.1:27018" },
    { _id: 2, host: "127.0.0.1:27019" }
  ]
})
```



Expected

```
{ ok: 1 }
```



Quick summary

Problem

mongosh not found

annoying path

Fix

run from extracted folder

add to PATH

Once you run `rs.initiate`, your cluster is officially DONE 🎉

If anything errors next, paste it 👍



```

test> rs.initiate({
  |_id:"rs0",
  | members: [
  | {_id: 0, host: "127.0.0.1:27017" },
  | {_id: 1, host: "127.0.0.1:27018" },
  | {_id: 2, host: "127.0.0.1:27019" }
  | ]
  | })
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778086739, i: 1 }),
    signature: {
      hash:
Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAA
AAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778086739, i: 1 })
}
rs0 [direct: secondary] test>

```

Nice — that output means your replica set **initialized successfully** 🎉

✅ What just happened

JSON

👉 Replica set created ✅

👉 All 3 nodes connected ✅

😬 Why you see this

```
rs0 [direct: secondary]
```

👉 You are currently connected to a node that is **SECONDARY**, not PRIMARY.

This is normal right after setup.

✅ Step 1: Find the PRIMARY

Run:

JavaScript

Look for:

```
"stateStr" : "PRIMARY"
```

👉 Note the port (27017 / 27018 / 27019)

✅ Step 2: Connect to PRIMARY

Example (if PRIMARY is 27018):

```
bat
```



Step 3: Verify role

Run:

```
JavaScript
```

(or newer command)

```
JavaScript
```



You should see:

```
JSON
```



Step 4: Test cluster (important)

On PRIMARY:

```
use mydb
```

```
JavaScript
```



Step 5: Check replication

Connect to another node:

```
bat
```

Then:

```
rs.slaveOk()  
use mydb
```

```
JavaScript
```

👉 You should see the same data ✔

What you achieved

You built a working **MongoDB Replica Set**, which gives:

- ✔ Automatic failover
- ✔ Data replication
- ✔ High availability

Bonus test (optional but cool)

Kill PRIMARY (Ctrl + C)

👉 Watch another node become PRIMARY automatically

Small note

That earlier warning:

```
Access control is not enabled
```

👉 Means:

- No authentication (fine for learning)
- Not secure for production



You're done

You now have a **fully working MongoDB cluster on one PC** — which is exactly how professionals test setups locally.

If you want next:

-  [Add authentication](#)
-  [Connect app to replica set](#)
-  [Convert to sharded cluster](#)

Just tell me 



```
rs0 [direct: secondary] test> rs.status()
{
  set: 'rs0',
  date: ISODate('2026-05-06T16:59:40.974Z'),
  myState: 1,
  term: Long('1'),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long('2000'),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1778086771,
i: 1 }), t: Long('1') },
    lastCommittedWallTime: ISODate('2026-05-
06T16:59:31.282Z'),
    readConcernMajorityOpTime: { ts: Timestamp({ t:
1778086771, i: 1 }), t: Long('1') },
    appliedOpTime: { ts: Timestamp({ t: 1778086771, i: 1 }), t:
Long('1') },
```



```
    durableOpTime: { ts: Timestamp({ t: 1778086771, i: 1 }), t:
Long('1') },
    writtenOpTime: { ts: Timestamp({ t: 1778086771, i: 1 }), t:
Long('1') },
    lastAppliedWallTime: ISODate('2026-05-
06T16:59:31.282Z'),
    lastDurableWallTime: ISODate('2026-05-
06T16:59:31.282Z'),
    lastWrittenWallTime: ISODate('2026-05-
06T16:59:31.282Z')
  },
  lastStableRecoveryTimestamp: Timestamp({ t:
1778086739, i: 1 }),
  electionCandidateMetrics: {
    lastElectionReason: 'electionTimeout',
    lastElectionDate: ISODate('2026-05-06T16:59:11.125Z'),
    electionTerm: Long('1'),
    lastCommittedOpTimeAtElection: { ts: Timestamp({ t:
1778086739, i: 1 }), t: Long('-1') },
    lastSeenWrittenOpTimeAtElection: { ts: Timestamp({ t:
1778086739, i: 1 }), t: Long('-1') },
    lastSeenOpTimeAtElection: { ts: Timestamp({ t:
1778086739, i: 1 }), t: Long('-1') },
    numVotesNeeded: 2,
    priorityAtElection: 1,
    electionTimeoutMillis: Long('10000'),
    numCatchUpOps: Long('0'),
    newTermStartDate: ISODate('2026-05-
06T16:59:11.233Z'),
    wMajorityWriteAvailabilityDate: ISODate('2026-05-
06T16:59:11.703Z')
  },
  members: [
    {
      _id: 0,
      name: '127.0.0.1:27017',
      health: 1,
      state: 1,
      stateStr: 'PRIMARY',
      uptime: 662,
```

```

    optime: { ts: Timestamp({ t: 1778086771, i: 1 }), t:
Long('1') },
    optimeDate: ISODate('2026-05-06T16:59:31.000Z'),
    optimeWritten: { ts: Timestamp({ t: 1778086771, i: 1 }),
t: Long('1') },
    optimeWrittenDate: ISODate('2026-05-
06T16:59:31.000Z'),
    lastAppliedWallTime: ISODate('2026-05-
06T16:59:31.282Z'),
    lastDurableWallTime: ISODate('2026-05-
06T16:59:31.282Z'),
    lastWrittenWallTime: ISODate('2026-05-
06T16:59:31.282Z'),
    syncSourceHost: "",
    syncSourceId: -1,
    infoMessage: 'Could not find member to sync from',
    electionTime: Timestamp({ t: 1778086751, i: 1 }),
    electionDate: ISODate('2026-05-06T16:59:11.000Z'),
    configVersion: 1,
    configTerm: 1,
    self: true,
    lastHeartbeatMessage: ""
  },
  {
    _id: 1,
    name: '127.0.0.1:27018',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 41,
    optime: { ts: Timestamp({ t: 1778086771, i: 1 }), t:
Long('1') },
    optimeDurable: { ts: Timestamp({ t: 1778086771, i: 1 }),
t: Long('1') },
    optimeWritten: { ts: Timestamp({ t: 1778086771, i: 1 }),
t: Long('1') },
    optimeDate: ISODate('2026-05-06T16:59:31.000Z'),
    optimeDurableDate: ISODate('2026-05-
06T16:59:31.000Z'),
    optimeWrittenDate: ISODate('2026-05-

```

```

06T16:59:31.000Z'),
  lastAppliedWallTime: ISODate('2026-05-
06T16:59:31.282Z'),
  lastDurableWallTime: ISODate('2026-05-
06T16:59:31.282Z'),
  lastWrittenWallTime: ISODate('2026-05-
06T16:59:31.282Z'),
  lastHeartbeat: ISODate('2026-05-06T16:59:39.394Z'),
  lastHeartbeatRecv: ISODate('2026-05-
06T16:59:40.359Z'),
  pingMs: Long('0'),
  lastHeartbeatMessage: '',
  syncSourceHost: '127.0.0.1:27017',
  syncSourceId: 0,
  infoMessage: '',
  configVersion: 1,
  configTerm: 1
},
{
  _id: 2,
  name: '127.0.0.1:27019',
  health: 1,
  state: 2,
  stateStr: 'SECONDARY',
  uptime: 41,
  optime: { ts: Timestamp({ t: 1778086771, i: 1 }), t:
Long('1') },
  optimeDurable: { ts: Timestamp({ t: 1778086771, i: 1 }),
t: Long('1') },
  optimeWritten: { ts: Timestamp({ t: 1778086771, i: 1 }),
t: Long('1') },
  optimeDate: ISODate('2026-05-06T16:59:31.000Z'),
  optimeDurableDate: ISODate('2026-05-
06T16:59:31.000Z'),
  optimeWrittenDate: ISODate('2026-05-
06T16:59:31.000Z'),
  lastAppliedWallTime: ISODate('2026-05-
06T16:59:31.282Z'),
  lastDurableWallTime: ISODate('2026-05-
06T16:59:31.282Z'),

```

```

lastWrittenWallTime: ISODate('2026-05-
06T16:59:31.282Z'),
lastHeartbeat: ISODate('2026-05-06T16:59:39.394Z'),
lastHeartbeatRecv: ISODate('2026-05-
06T16:59:40.316Z'),
pingMs: Long('0'),
lastHeartbeatMessage: "",
syncSourceHost: '127.0.0.1:27017',
syncSourceId: 0,
infoMessage: "",
configVersion: 1,
configTerm: 1
}
],
ok: 1,
'$clusterTime': {
  clusterTime: Timestamp({ t: 1778086771, i: 1 }),
  signature: {
    hash:
Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAA
AAAAA=', 0),
    keyId: Long('0')
  }
},
operationTime: Timestamp({ t: 1778086771, i: 1 })
}
rs0 [direct: primary] test>

```

That output is exactly what a healthy replica set should look like 

Let's break it down so you fully understand what you built.



Your cluster is WORKING

Key part from your output:

```

stateStr: 'PRIMARY' → 127.0.0.1:27017
stateStr: 'SECONDARY' → 127.0.0.1:27018

```

```
stateStr: 'SECONDARY' → 127.0.0.1:27019
```

👉 This means:

- ● 1 PRIMARY (main node handling writes)
- ● 2 SECONDARY (replicating data)

- ✓ Replica set is healthy
- ✓ All nodes are connected
- ✓ Replication is active

🧠 Important details (from your status)

✓ Majority is correct

```
majorityVoteCount: 2
```

👉 Out of 3 nodes, 2 are needed → correct

✓ Replication is working

```
syncSourceHost: '127.0.0.1:27017'
```

👉 Both secondaries are syncing from PRIMARY

✓ All nodes healthy

```
health: 1
```

👉 Means all nodes are alive

🚀 Now test it (don't skip)

● Step 1: Insert data (on PRIMARY)

You are already connected to PRIMARY:

```
rs0 [direct: primary]
```

Run:

```
use mydb
```

JavaScript

Step 2: Read from SECONDARY

Open new terminal:

```
bat
```

Then:

```
rs.slaveOk()
```

```
use mydb
```

JavaScript

👉 You should see the same document ✔

Bonus: Test failover (this proves it's real cluster)

Kill PRIMARY (Ctrl + C on 27017 terminal)

👉 Wait ~5–10 seconds

Then check another node:

JavaScript

👉 One of these will become:

PRIMARY

✨ That's automatic failover working



Small note

This message:

```
Access control is not enabled
```



Means:

- No username/password
- OK for learning
- Not safe for real apps



What you achieved (important)

You built a real **MongoDB Replica Set** in MongoDB

Which gives:

- High availability
- Data replication
- Automatic failover



If you want next level

I can guide you to:

-  Add authentication (real-world setup)
-  Connect Node.js / Java app
-  Convert this into **Sharded Cluster**
-  Monitor cluster



Right now: **You've successfully completed MongoDB clustering on a single PC** — exactly what you wanted.



```
C:\Program Files\MongoDB\Server\8.3\bin> mongosh --  
port 27018  
'mongosh' is not recognized as an internal or external  
command,  
operable program or batch file.
```

```
C:\Program Files\MongoDB\Server\8.3\bin>
```

Yep — same issue again: `mongosh` isn't in this folder or PATH 👍

Your cluster is fine; this is just how MongoDB packages the shell separately.

✅ Quickest fix (no setup needed)

👉 Don't use `mongosh` directly here.

Run it with **full path** to where you extracted it.

Example:

```
bat
```

👉 Replace `YourName` and folder name accordingly.

✅ Better fix (recommended once)

Add `mongosh` to PATH so you can use it anywhere.

Steps:

1. Go to your `mongosh` folder

Example:


```
C:\Users\YourName\Downloads\mongosh-2.8.3-win32-x64\bin
```

2. Copy that path
3. Search Environment Variables
4. Open Edit system environment variables
5. Click Environment Variables
6. Select Path → Edit → New
7. Paste the path → OK

After that, restart CMD and run:

```
bat
```



What you're trying to do

You're connecting to:

```
127.0.0.1:27018 (SECONDARY)
```

Then run:

```
rs.slaveOk()  
use mydb
```

JavaScript

👉 This will prove replication is working ✔️



Tip (pro-level)

You can connect to the whole cluster at once:

bat

👉 This is how real apps connect.

If you want, next step I can show you:

👉 [how to connect this cluster to a Node.js app or GUI \(MongoDB Compass\)](#)

📄 👍 💬 ↗️ ↺️ ⋮

```
rs0 [direct: secondary] test> rs.slaveOk()  
TypeError: rs.slaveOk is not a function  
rs0 [direct: secondary] test>
```

Good catch — this is **not your mistake**, it's a version change 👍

! Why this error happens

```
TypeError: rs.slaveOk is not a function
```

👉 In newer versions of [MongoDB](#) (and [mongosh](#)),
`rs.slaveOk()` is **removed / deprecated**

✅ Correct command (use this instead)

Run:

JavaScript

✅ Then test replication

```
use mydb
```

JavaScript

👉 You should now see the data from PRIMARY ✔



What this does

- `setReadPref("secondary")` → allows reads from secondary nodes
- By default:
 - PRIMARY = read/write
 - SECONDARY = no reads (unless enabled)



Alternative (modern way)

You can also connect like this:

Bash

👉 Then you don't need to set it manually



Summary

Old command

```
rs.slaveOk() ❌
```

New command

```
db.getMongo().setReadPref("secondary") ✔
```



Next check

After running, confirm:

JavaScript

👉 If data appears → your cluster is **fully working** 🎉

If you want next:

- simulate failover
- add authentication
- connect app

Just tell me 👍



done

Nice — your replica set is fully up and healthy 👍

You've got:

- 1 **PRIMARY** (27017)
- 2 **SECONDARY** nodes (27018, 27019)
- replication working correctly

That's exactly what you want.

About your last issues

1. mongosh not recognized

That just means the MongoDB shell isn't in your PATH.

Instead of fixing PATH right now, just run it directly:

Bash

If that fails, check if `mongosh.exe` exists in that folder.
(Newer installs sometimes separate it.)

2. `rs.slaveOk()` error

Correct — that command is **deprecated/removed**.

Use this instead:

```
JavaScript
```

What to do next (important)

Now that your replica set is running, here are the real next steps:

✓ 1. Test replication

On PRIMARY (27017):

```
use test
```

```
JavaScript
```

Then connect to a secondary (27018) and run:

```
db.getMongo().setReadPref("secondary")  
use test
```

```
JavaScript
```

👉 You should see the same data.

✓ 2. Test failover (very important)

Kill the PRIMARY process:

Bash

Then watch election from another node:

JavaScript

👉 One secondary should become PRIMARY automatically.

✅ 3. Restart the killed node

Start it again:

Bash

👉 It should rejoin as SECONDARY.

✅ 4. (Optional but recommended) Add authentication

Right now your cluster is **open (no security)**.

Next step if you want production-like setup:

- Create admin user
- Enable `security.authorization: enabled`
- Use `keyFile` for replica set auth

✅ 5. (Optional) Fix PATH permanently

So you can run `mongosh` anywhere:

Bash

Restart terminal after that.

Final status

You've successfully:

- Installed MongoDB
- Created 3-node cluster
- Initialized replica set
- Verified replication

That's already **intermediate-level MongoDB work**.

If you want, next I can guide you through:

-  Secure replica set (authentication + keyfile)
-  Connect from Node.js / Python
-  Read/write concerns (majority, etc.)
-  Sharding setup (next level)

Just tell me 

